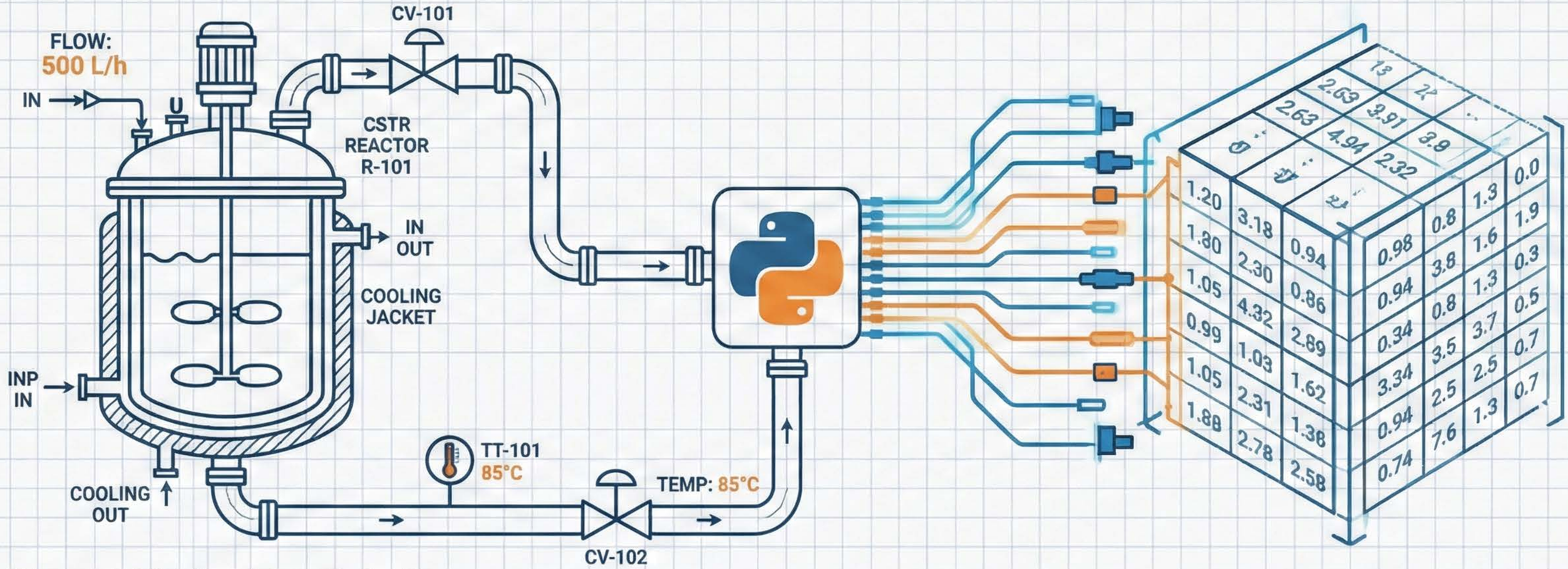
Unit 03 NumPy - 9. 實務應用案例

連接化工原理與數值運算的橋梁

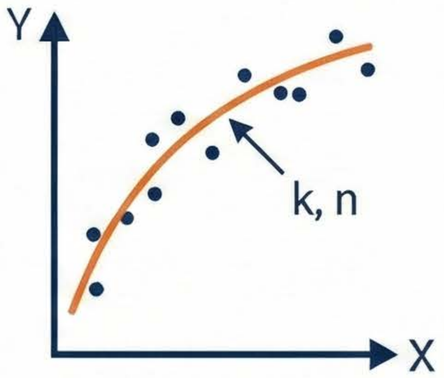


- 從理論推導到程式實作
- 本節重點：如何將反應器、熱傳導與質量平衡問題轉化為 NumPy 陣列運算。

NotebookLM

數值運算的三大化工應用場景

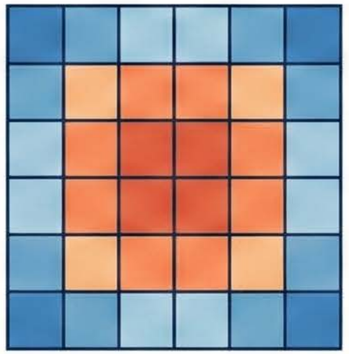
反應動力學 (Kinetics)



任務：數據擬合

從實驗數據估算反應率常常數 k 與級數 n。

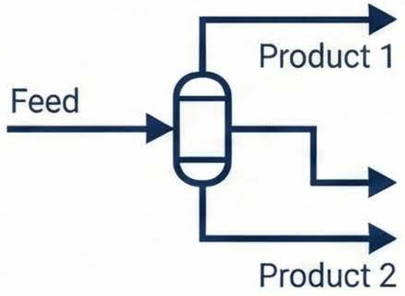
熱傳模擬 (Heat Transfer)



任務：網格計算

使用二維陣列模擬平板溫分佈 (Finite Difference Method)。

質量平衡 (Mass Balance)



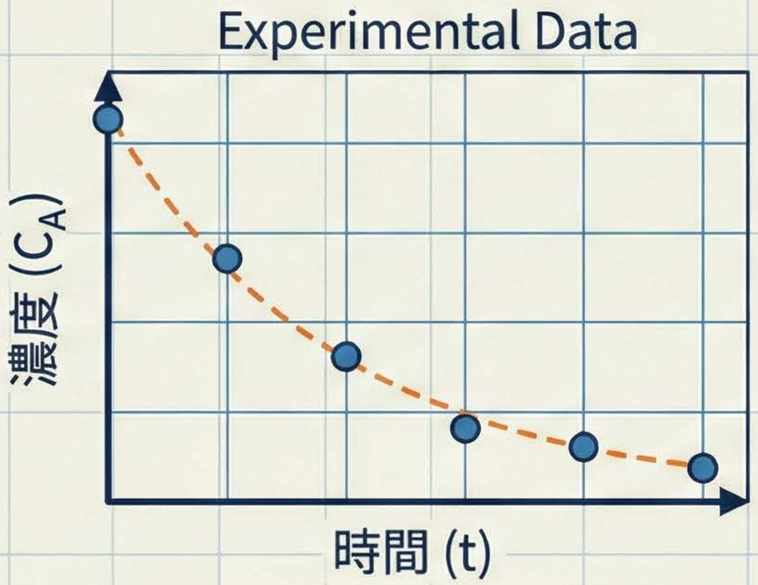
任務：線性代數

解析複雜單元操作流程中的多變數聯立方程組 (Ax=B)。

NotebookLM

案例一：反應動力學參數估算 (Reaction Kinetics)

1. 物理問題 (The Physical Problem)



實驗室測得批次反應器中濃度隨時間變化的數據點。

Linearization

2. 數學模型 (The Mathematical Model)

$$-\frac{dC_A}{dt} = kC_A$$

$$\ln(C_A) = \ln(C_{A0}) - kt$$

目標：利用線性迴歸找出斜率 (Slope)，即為 -k。

NotebookLM

數位實作：向量化數據處理 (Digital Implementation)

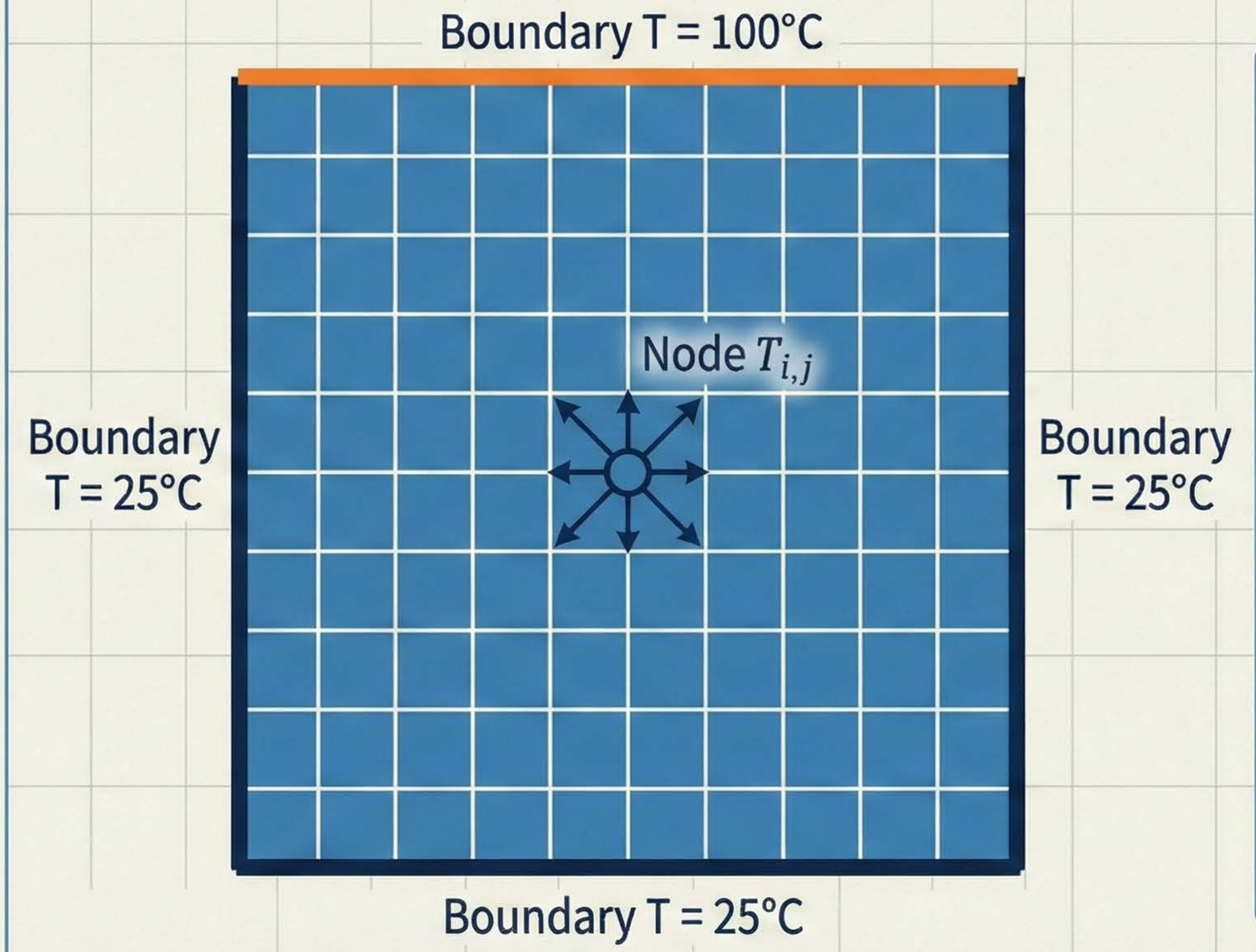
time	0	10	20	30
conc_A	1.0	0.8	0.65	0.5

```
# 1. 定義數據 (ndarray)
time = np.array([0, 10, 20, 30])
conc_A = np.array([1.0, 0.8, 0.65, 0.5])

# 2. 向量化運算 (Vectorization)
ln_C = np.log(conc_A)  # 自動對所有元素取對數

# 3. 線性擬合求解
k, intercept = np.polyfit(time, ln_C, 1)
print(f"Rate Constant k: {-k:.4f}")
```

Vectorization (向量化)：一行程式碼處理整個陣列，取代 For 迴圈。

NotebookLM

案例二：二維熱傳導模擬 (2D Heat Transfer Simulation)



物理模型

平板邊界恒溫，求穩態時的內部溫度分佈。

數值方法

有限差分法 (Finite Difference Method)。

網格化 (Discretization)

將連續空間切割為 $N \times N$ 的離散點。

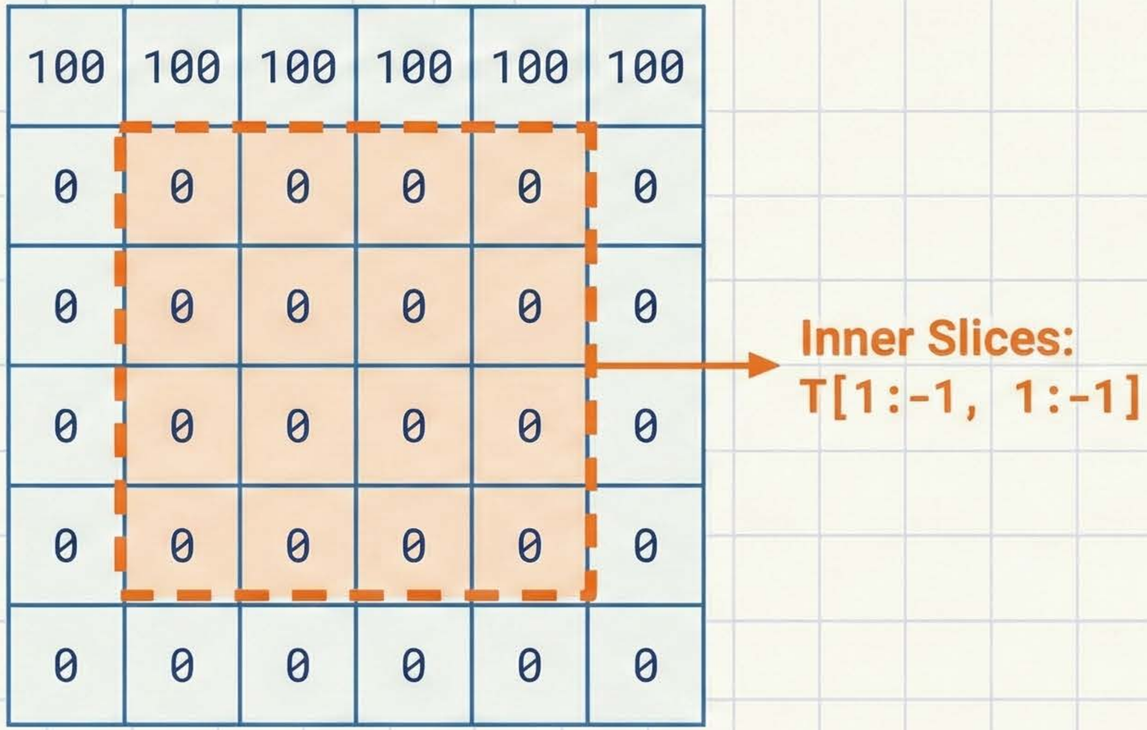
核心算法

每一點的溫度等於其上下左右四點的平均值。

$$T_{i,j} = \frac{1}{4}(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1})$$

NotebookLM

數位實作：網格與切片運算 (Grid & Slicing)

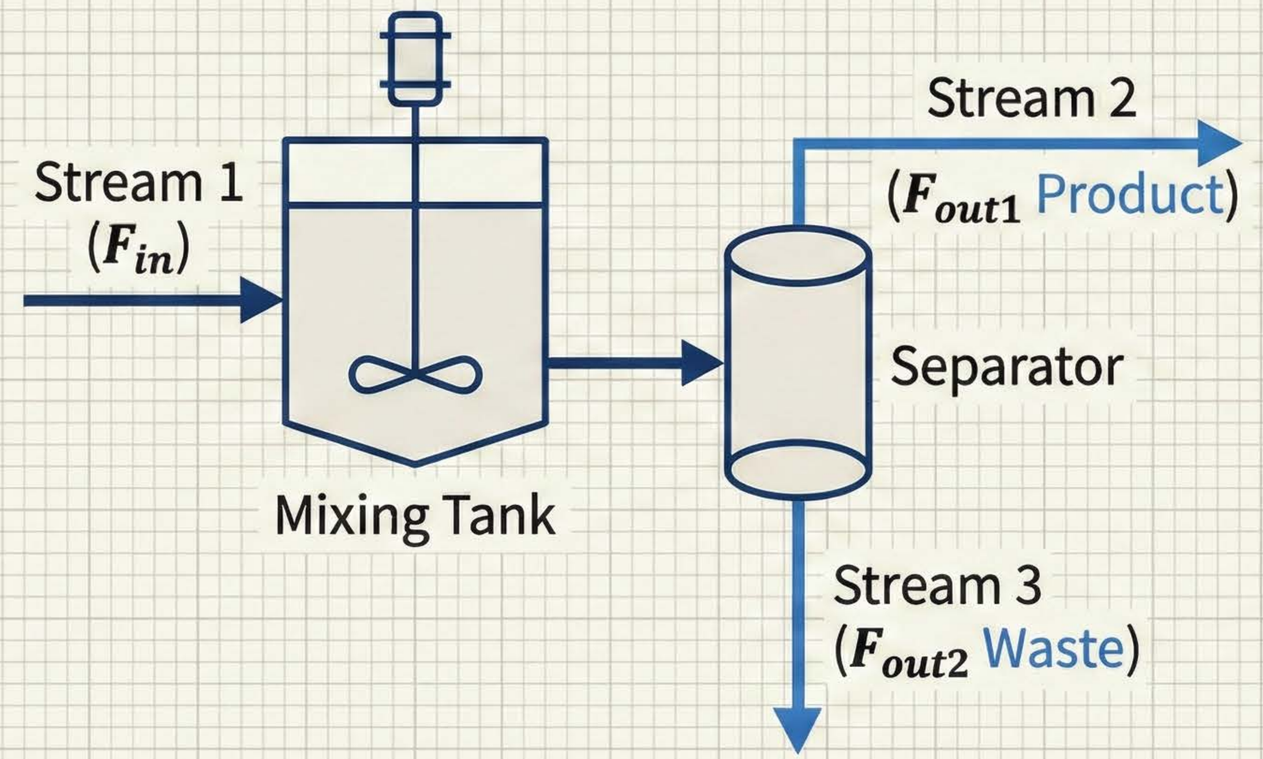



```python
# 初始化 10x10 網格
T = np.zeros((10, 10))
T[0, :] = 100  # 設定上邊界溫度

# 迭代更新 (使用切片)
for i in range(1000):
    T[1:-1, 1:-1] = 0.25 * (
        T[0:-2, 1:-1] +  # 上鄰居
        T[2:,   1:-1] +  # 下鄰居
        T[1:-1, 0:-2] +  # 左鄰居
        T[1:-1, 2:]      # 右鄰居
    )
```

Slicing Advantage: 利用切片語法同時更新所有內部網格點，大幅提升計算效能，避免使用慢速的 Python 雙重迴圈。

NotebookLM

案例三：質量平衡計算 (Mass Balance Calculation)



建立聯立方程組
(System of Linear Equations)

總質量平衡 (Total Balance)：

$$F_{in} = F_{out1} + F_{out2}$$

組分 A 平衡 (Component A Balance)：

$$x_{in}F_{in} = x_1F_{out1} + x_2F_{out2}$$

Challenge

當製程變得複雜（如包含 10+ 單元操作與回收流）時，手動求解極為困難。

轉化為矩陣形式

$$[A][X] = [B]$$

Industrial Blueprint meets Swiss Style

數位實作：線性代數求解器 (Linear Algebra Solver)

1. 聯立方程式 (System of Equations)

$$x + y = 100$$
$$0.2x + 0.5y = 30$$

2. 矩陣形式 (Matrix Form)

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 0.2 & 0.5 \end{bmatrix}}_{[A]} \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_{[X]} = \underbrace{\begin{bmatrix} 100 \\ 30 \end{bmatrix}}_{[B]}$$

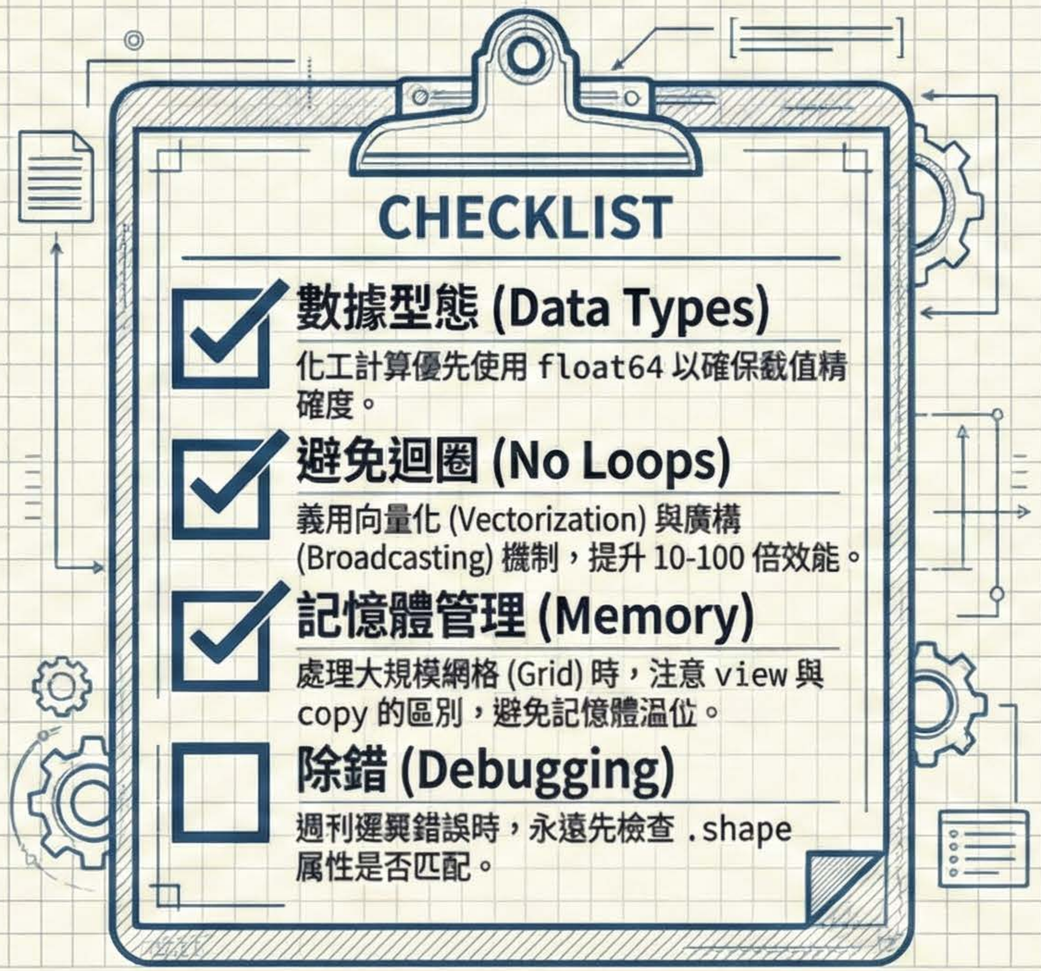
Python NumPy Implementation

```python
import numpy as np

# 1. 定義係數矩陣 A 與常數向量 B
A = np.array([[1, 1], [0.2, 0.5]])
B = np.array([100, 30])

# 2. 呼叫求解器 (Solver)
solution = np.linalg.solve(A, B)

print(f"Flow Rates: {solution}")
```

One Line Solution: 底層使用 LAPACK 優化庫，極速求解大規模系統。

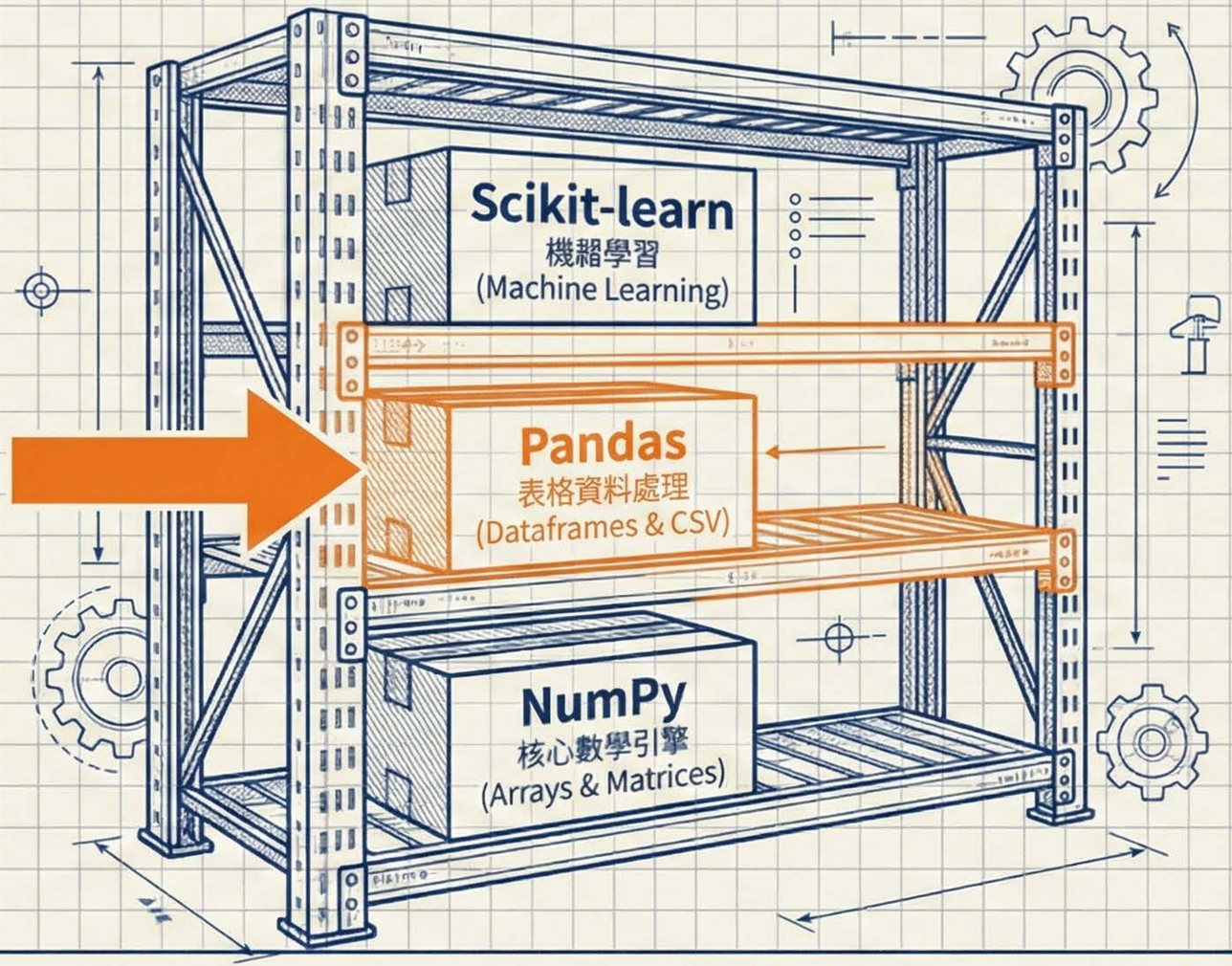
NotebookLM

最佳實務總結 (Best Practices & Summary)



“Code is read much more often than it is written.”

下一步：從數值陣列到結構化數據



您已經掌握了化工數值計算的核心引擎 (NumPy)。

接下來，我們將進入 Unit 04，學習如何使用 **Pandas** 處理更複雜的製程大數據與時間序列。

Unit 03 結束 | 前往 Unit 04 Pandas 資料處理

NotebookLM